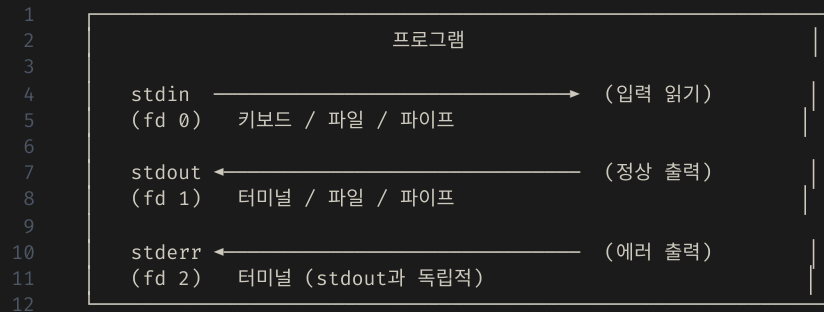


C++ 프로그래밍

표준 스트림 — `stdin` / `stdout` / `stderr`

세 가지 표준 스트림

모든 프로그램은 시작할 때 운영체제로부터 세 개의 스트림을 자동으로 받는다.



스트림	파일 기술자	C	C++	기본 방향
stdin	0	scanf , fgets , getchar	std::cin	키보드
stdout	1	printf , puts	std::cout	터미널
stderr	2	fprintf(stderr, ...) , perror	std::cerr	터미널

stdout은 버퍼링되고, stderr는 버퍼링 없이 즉시 출력된다.

C vs C++ 표준 입출력 개요

C 스타일

```
1  #include <stdio.h>
2
3  int main(void) {
4      int age;
5      char name[32];
6
7      // stdout 출력
8      printf("이름을 입력하세요: ");
9
10     // stdin 입력
11     scanf("%s", name);
12     scanf("%d", &age);
13
14     printf("안녕하세요, %s님! 나이: %d\n",
15           name, age);
16     return 0;
17 }
```

- 형식 지정자 (%d , %s , ...) 직접 작성
- 타입 불일치 시 런타임 오류 (컴파일 오류 아님)
- scanf 에 & 연산자 필요 (포인터 전달)

C++ 스타일

```
1  #include <iostream>
2  #include <string>
3
4  int main() {
5      int age;
6      std::string name;
7
8      // stdout 출력
9      std::cout << "이름을 입력하세요: ";
10
11     // stdin 입력
12     std::cin >> name >> age;
13
14     std::cout << "안녕하세요, " << name
15               << "님! 나이: " << age << "\n";
16     return 0;
17 }
```

- 형식 지정자 없음 — 타입을 자동으로 인식
- 타입 불일치 시 컴파일 오류 (더 안전)
- & 연산자 불필요

std::cout — 기본 출력

<< 연산자(삽입 연산자)로 값을 출력 스트림에 밀어 넣는다.

기본 사용

```
1 #include <iostream>
2 #include <string>
3
4 int main() {
5     int    n = 42;
6     double d = 3.14;
7     bool   b = true;
8     std::string s = "hello";
9
10    std::cout << n << "\n";    // 42
11    std::cout << d << "\n";    // 3.14
12    std::cout << b << "\n";    // 1 (기본값)
13    std::cout << s << "\n";    // hello
14
15    // 연결(chaining) - 한 줄에 여러 값 출력
16    std::cout << "n=" << n
17                << ", d=" << d << "\n";
18 }
```

"\n" VS std::endl

```
std::cout << "줄 바꿈\n";    // ✅ 빠름
std::cout << "줄 바꿈" << std::endl; // ⚠️ flush 포함 - 느림
// endl은 버퍼를 즉시 비움 → 성능 저하
// 일반적으로 "\n" 사용 권장
```

using namespace std

```
// std:: 접두사를 생략할 수 있음
using namespace std;

cout << "hello\n";    // std::cout 대신
cin >> n;             // std::cin 대신

// ⚠️ 헤더 파일에서는 사용하지 말 것
// 이름 충돌을 일으킬 수 있음
```

printf VS std::cout 핵심 차이

```
int n = 42; double d = 3.14159;

// printf: 형식 지정자 필요
printf("%.2f\n", d);
printf("%10d\n", n);

// cout: 조작자로 서식 제어
std::cout << std::fixed
           << std::setprecision(2) << d << "\n";
std::cout << std::setw(10) << n << "\n";

// ❌ printf 타입 불일치 → 런타임 UB
printf("%d\n", d);
// ✅ cout 타입 불일치 → 컴파일 오류
```

std::cout — 서식 조작자 (<iomanip>)

printf 의 형식 지정자에 대응하는 C++의 출력 서식 제어 도구다.

숫자 서식

```
1  #include <iostream>
2  #include <iomanip>
3
4  int main() {
5      double pi = 3.141592653589793;
6      int n = 255;
7
8      // 소수점 자릿수
9      std::cout << std::fixed
10         << std::setprecision(2) << pi << "\n"; // 3.14
11      std::cout << std::setprecision(5) << pi << "\n"; // 3.14159
12
13      // 진법 출력
14      std::cout << std::dec << n << "\n"; // 255 (10진수)
15      std::cout << std::hex << n << "\n"; // ff (16진수)
16      std::cout << std::oct << n << "\n"; // 377 (8진수)
17      std::cout << std::hex << std::uppercase
18         << n << "\n"; // FF (대문자)
19
20      // 부호 강제 출력
21      std::cout << std::showpos << 42 << "\n"; // +42
22 }
```

폭(width)과 정렬

```
1  #include <iostream>
2  #include <iomanip>
3
4  int main() {
5      // setw: 다음 출력 한 번에만 적용
6      std::cout << std::setw(10) << 42 << "\n";
7      // "          42" (오른쪽 정렬, 기본)
8
9      std::cout << std::left
10         << std::setw(10) << "hi" << "\n";
11      // "hi      |" (왼쪽 정렬)
12
13      // 빈 자리 채우기
14      std::cout << std::setfill('0')
15         << std::setw(6) << 42 << "\n"; // 000042
16
17      // 표 형식 출력
18      std::cout << std::left
19         << std::setw(12) << "이름"
20         << std::setw(6) << "나이"
21         << std::setw(8) << "점수" << "\n";
22      std::cout << std::setw(12) << "홍길동"
23         << std::setw(6) << 20
24         << std::setw(8) << 95.5 << "\n";
25 }
```

stderr — 에러 스트림

정상 출력(stdout)과 에러 출력(stderr)을 분리하면 리다이렉션으로 각각 따로 처리할 수 있다.

C — fprintf(stderr, ...) / perror

```
1 #include <stdio.h>
2 #include <errno.h> // errno
3 #include <string.h> // strerror
4
5 int main(void) {
6     FILE* fp = fopen("없는파일.txt", "r");
7     if (fp == NULL) {
8         // fprintf(stderr, ...): 에러 스트림에 직접 출력
9         fprintf(stderr, "오류: 파일을 열 수 없음\n");
10        // perror: errno를 해석해 자동으로 stderr에 출력
11        perror("fopen 실패");
12        // 출력 예: "fopen 실패: No such file or directory"
13        // strerror: errno 코드를 문자열로 변환
14        fprintf(stderr, "errno %d: %s\n",
15                errno, strerror(errno));
16        return 1;
17    }
18    fclose(fp);
19    return 0;
20 }
```

- `perror(msg) = fprintf(stderr, "%s: %s\n", msg, strerror(errno))`
- `errno` : 마지막 시스템 호출의 오류 코드 (전역 변수)

C++ — std::cerr / std::clog

```
1 #include <iostream>
2 #include <cerrno> // errno
3 #include <cstring> // strerror
4 #include <fstream>
5
6 int main() {
7     std::ifstream file("없는파일.txt");
8     if (!file) {
9         // cerr: 버퍼링 없이 즉시 출력 (unbuffered)
10        std::cerr << "오류: 파일을 열 수 없음\n";
11        // errno 활용
12        std::cerr << "errno " << errno
13                << ": " << std::strerror(errno) << "\n";
14    }
15    // clog: 버퍼링 있는 에러 스트림 (진단 로그용)
16    std::clog << "[DEBUG] 파일 열기 시도\n";
17 }
```

스트림	버퍼링	용도
<code>std::cout</code>	있음	정상 출력
<code>std::cerr</code>	없음	즉시 에러 출력
<code>std::clog</code>	있음	진단.로그 출력

std::cin — 기본 입력

>> 연산자(추출 연산자)로 입력 스트림에서 값을 읽는다.

기본 사용

```
1  #include <iostream>
2  #include <string>
3
4  int main() {
5      int    age;
6      double score;
7      std::string name;
8
9      // 공백/줄바꿈으로 구분된 값 읽기
10     std::cin >> age;           // 정수 읽기
11     std::cin >> score;         // 실수 읽기
12     std::cin >> name;          // 단어 읽기 (공백 전까지)
13
14     // 연결(chaining)
15     std::cin >> age >> score >> name;
16
17     // 입력 성공 여부 확인
18     if (!(std::cin >> age)) {
19         std::cerr << "잘못된 입력\n";
20         std::cin.clear();           // 에러 플래그 초기화
21         std::cin.ignore(1000, '\n'); // 버퍼 비우기
22     }
23 }
```

scanf VS std::cin

```
// C: scanf - 형식 지정자 + & 연산자 필요
int a; double b;
scanf("%d %lf", &a, &b);

// ❌ & 빠뜨리면 런타임 크래시
scanf("%d", a); // *a 에 쓰려 함 → UB

// C++: cin - & 불필요, 타입 자동 인식
int a; double b;
std::cin >> a >> b;
```

>> 직후 getline 주의

```
1  #include <iostream>
2  #include <string>
3
4  int main() {
5      int age;
6      std::string name;
7
8      std::cin >> age;
9      // ⚠ 버퍼에 '\n' 이 남음
10
11     std::cin.ignore();           // '\n' 제거
12     std::getline(std::cin, name); // ✅ 정상 동작
13 }
```

std::cin — 한 줄 입력 (getline)

>> 연산자는 공백에서 멈춘다. 공백 포함 한 줄을 읽으려면 std::getline 을 사용한다.

>> VS getline

```
1  #include <iostream>
2  #include <string>
3
4  int main() {
5      std::string s;
6
7      // >> 는 공백 전까지만 읽음
8      std::cin >> s;
9      // 입력: "홍길동"
10     // s = "홍" (길동은 버퍼에 남음)
11
12     // getline은 줄 전체 읽음
13     std::getline(std::cin, s);
14     // 입력: "홍길동"
15     // s = "홍길동" ✅
16 }
```

EOF까지 모든 줄 읽기

```
1  #include <iostream>
2  #include <string>
3
4  int main() {
5      std::string line;
6      while (std::getline(std::cin, line)) {
7          std::cout << line << "\n";
8      }
9      // 파일 리다이렉션이나 파이프로 전달된
10     // 모든 내용을 처리할 때 유용
11 }
```

scanf 한 줄 읽기 vs getline

// C: 크기 제한 필요, 문법 복잡
char buf[64];
scanf(" %63[^\n]", buf); // 공백 먹고 줄 끝까지
// ⚠ 버퍼 크기 초과 시 UB

// C (fgets - 더 안전)
fgets(buf, sizeof(buf), stdin);
// '\n'이 버퍼에 포함될 수 있음 (직접 제거 필요)

// C++: 간단하고 안전
std::string line;
std::getline(std::cin, line); // 크기 제한 없음 ✅

커맨드라인 스트림 리다이렉션

셸에서 `<`, `>`, `2>` 기호로 스트림의 방향을 파일이나 다른 프로그램으로 바꿀 수 있다.

⚠ 주의: Windows 환경에서는 스트림 리다이렉션을 **커맨드 프롬프트(cmd)**에서 수행하세요. 커맨드 프롬프트에서는 POSIX 계열 운영체제와 유사하게 동작합니다. PowerShell에서는 리다이렉션 문법이 다르므로, 다음 예제가 그대로 동작하지 않을 수 있습니다.

기본 리다이렉션

```
# stdout을 파일로 저장 (보여쓰기)
./prog > output.txt

# stdout을 파일에 추가 (append)
./prog >> output.txt

# stdin을 파일에서 읽기
./prog < input.txt

# stderr를 파일로 저장
./prog 2> error.txt

# stdout과 stderr 모두 같은 파일로
./prog > output.txt 2>&1

# stderr만 버리기 (Linux/macOS)
./prog 2> /dev/null

# stdout과 stderr 모두 버리기
./prog > /dev/null 2>&1
```

파이프 (|)

```
# 첫 번째 프로그램의 stdout을
# 두 번째 프로그램의 stdin으로 연결
./producer | ./consumer

# 여러 프로그램 연결
./prog | sort | uniq > result.txt

# stdin/stdout/stderr 모두 제어
./prog < in.txt > out.txt 2> err.txt
```

프로그램 내에서 활용

```
1  #include <iostream>
2  #include <string>
3
4  int main() {
5      std::string line;
6      // ./prog < data.txt 로 실행하면
7      // 파일의 내용이 cin으로 들어옴
8      while (std::getline(std::cin, line)) {
9          std::cout << "[처리됨] " << line << "\n";
10     }
11 }
```

리다이렉션 실습 예제

⚠ 주의: Windows 환경에서는 스트림 리다이렉션을 **커맨드 프롬프트(cmd)**에서 수행하세요. 커맨드 프롬프트에서는 POSIX 계열 운영체제와 유사하게 동작합니다. PowerShell에서는 리다이렉션 문법이 다르므로, 다음 예제가 그대로 동작하지 않을 수 있습니다.

```
1 // grader.cpp - stdin에서 점수를 읽어 stdout/stderr로 분리 출력
2 #include <iostream>
3 #include <string>
4
5 int main() {
6     std::string name;
7     int score;
8
9     while (std::cin >> name >> score) {
10         if (score < 0 || score > 100)
11             std::cerr << "[오류] " << name << ": 범위 초과 점수 " << score << "\n"; // 잘못된 데이터 → stderr
12         else
13             std::cout << name << " " << (score >= 60 ? "합격" : "불합격") << "\n"; // 정상 데이터 → stdout
14     }
15 }
```

```
# input.txt 내용:
# 홍길동 85
# 이순신 110
# 강감찬 55

./grader < input.txt # stdout + stderr 모두 터미널에 출력
./grader < input.txt > result.txt # 정상 결과만 파일로
./grader < input.txt 2> errors.txt # 에러만 파일로
./grader < input.txt > result.txt 2> errors.txt # 각각 분리
```

stdout과 stderr를 처음부터 **분리해서 작성**하면 리다이렉션으로 유연하게 처리할 수 있다.

printf / scanf VS std::cout / std::cin 비교표

항목	printf / scanf	std::cout / std::cin
헤더	<cstdio>	<iostream>
형식 지정자	직접 작성 (%d , %f , ...)	불필요 — 타입 자동 인식
타입 안전성	❌ 런타임에서야 오류 발견	✅ 컴파일 타임 오류
scanf & 연산자	필요 (&a)	불필요 (cin >> a)
에러 출력	fprintf(stderr, ...) / perror	std::cerr <<
사용자 정의 타입 출력	불가	✅ operator<< 오버로딩
공백 포함 문자열 입력	fgets / scanf("%[^\n]", ...)	std::getline(cin, s)
성능	빠름	기본적으로 약간 느림*
서식 제어	형식 지정자 인라인	<iomanip> 조작자 사용

* std::ios::sync_with_stdio(false); std::cin.tie(nullptr); 를 main 시작에 추가하면 printf / scanf 와 동등한 속도를 낼 수 있다. 단, 이후 printf / scanf 와 혼용 금지.

요약

```
#include <iostream>    // cout, cin, cerr, clog
#include <iomanip>      // setw, setprecision, setfill, ...
#include <string>       // std::string + getline

// stdout 출력
std::cout << 값 << "\n";
std::cout << std::fixed << std::setprecision(2) << 3.14 << "\n";

// stderr 출력 (에러)
std::cerr << "에러 메시지\n";           // C++
fprintf(stderr, "에러: %s\n", msg);       // C
perror("시스템 호출 실패");              // C - errno 자동 해석

// stdin 입력
std::cin >> 변수 >> 변수;                // 공백 구분
std::getline(std::cin, str);              // 한 줄 전체
std::cin.ignore();                       // >> 후 getline 전 '\n' 제거

# 커맨드라인 리다이렉션
./prog < input.txt                      # stdin ← 파일
./prog > output.txt                     # stdout → 파일
./prog 2> error.txt                     # stderr → 파일
./prog > out.txt 2>&1                    # stdout + stderr → 같은 파일
./prog < in.txt | ./next                 # stdout → 다음 프로그램의 stdin
```

상황	권장
정상 출력	<code>std::cout</code> / <code>printf</code>
에러·경고 출력	<code>std::cerr</code> / <code>fprintf(stderr, ...)</code>
시스템 에러 출력	<code>perror(msg)</code> C 언어 표준 라이브러리 함수
공백 포함 한 줄 입력	<code>std::getline(std::cin, str)</code>
파일 전체 줄 처리	<code>while (std::getline(std::cin, line))</code>

C++ 프로그래밍

표준 스트림 버퍼와 파일 디스크립터

버퍼(Buffer)란?

프로그램이 출력을 요청할 때마다 즉시 OS에 시스템 콜을 보내면 **비효율적**이다.
버퍼는 데이터를 모았다가 **한 번에** 전달하는 중간 저장소다.

```
1  프로그램
2  |
3  | printf("A")   ← 시스템 콜 없음
4  | printf("B")   ← 시스템 콜 없음
5  | printf("C\n") ← \n 검출 → 버퍼 플러시 → 시스템 콜
6  | ↓
7  [ 유저 공간 버퍼 ] → 시스템 콜 → [ OS 커널 ] → 터미널/파일
```

버퍼 있음 (기본)

여러 번의 `printf` 를 모아 한 번에 전달

I/O 횟수 감소 → **성능 향상**

`stdout`은 **버퍼링**, `stderr`는 **버퍼링 없음** — 이유는 뒤에서 설명한다.

버퍼 없음

`fprintf(stderr, ...)` → 즉시 전달

항상 즉시 출력 → **신뢰성**

버퍼링의 세 가지 종류

종류	플러시 시점	적용 대상
풀 버퍼링 (Fully Buffered)	버퍼가 가득 찰 때	stdout → 파일/파이프
라인 버퍼링 (Line Buffered)	<code>\n</code> 을 만날 때	stdout → 터미널, stdin
언버퍼드 (Unbuffered)	즉시	stderr

```
1 // 라인 버퍼링 (터미널)
2 printf("loading..."); // \n 없음 → 화면에 아직 안 나옴
3 do_heavy_work();
4 printf("done\n"); // \n → 두 문자열이 한꺼번에 출력
5
6 // 풀 버퍼링 (파일 리다이렉션: ./prog > out.txt)
7 printf("Hello"); // 버퍼에만 저장
8 printf("World"); // 버퍼에만 저장
9 // 프로그램 종료 → fflush() 자동 호출 → 파일에 기록됨
10
11 // 언버퍼드 (stderr)
12 fprintf(stderr, "Error"); // 즉시 출력, 항상
```

같은 `printf` 라도 실행 환경(터미널.파일.파이프)에 따라 버퍼링 방식이 바뀐다.

stdout — 터미널 vs 파이프/파일

버퍼링 방식은 stdout이 어디에 연결되는지에 따라 자동으로 결정된다.

터미널 연결 (라인 버퍼링)

```
1 // ./prog 직접 실행
2 printf("Step 1\n"); // \n → 즉시 화면 출력
3 printf("Step 2\n"); // \n → 즉시 화면 출력
4
5 // ▲ \n 없으면 나중에 출력될 수 있음
6 printf("loading...");
7 // (무거운 작업 진행 중)
8 // "loading..." 이 작업 중에 보이지 않을 수 있음
9 // fflush(stdout)으로 해결
10 printf(" done\n");
```

파이프/파일 연결 (풀 버퍼링)

```
./prog | cat # stdout → 파이프 → 풀 버퍼링
./prog > out.txt # stdout → 파일 → 풀 버퍼링
```

```
1 // 풀 버퍼링 모드에서는 \n도 즉시 플래시하지 않음
2 for (int i = 0; i < 10; i++) {
3     printf("%d\n", i); // 버퍼에 쌓임
4 }
5 // 프로그램 종료 시 한꺼번에 기록
```

버퍼 모드 강제 변경

```
1 #include <stdio.h>
2
3 // 프로그램 시작 직후, 첫 I/O 전에 호출
4 int main() {
5     // stdout을 강제로 언버퍼드로
6     setvbuf(stdout, NULL, _IONBF, 0);
7
8     // stdout을 라인 버퍼링으로
9     setvbuf(stdout, NULL, _IOLBF, 0);
10
11     // stdout을 풀 버퍼링으로 (버퍼 크기 지정)
12     setvbuf(stdout, NULL, _IOFBF, 4096);
13 }
```

상수

모드

`_IONBF`

언버퍼드

`_IOLBF`

라인 버퍼링

`_IOFBF`

풀 버퍼링

파이프/파일 환경에서도 실시간 출력이 필요하다면

`setvbuf(stdout, NULL, _IOLBF, 0)` 또는 `fflush(stdout)` 을 사용한

stderr — 언버퍼드인 이유

stderr가 버퍼 없이 즉시 출력되는 이유는 프로그램 크래시 시에도 메시지가 유실되지 않아야 하기 때문이다.

크래시 시나리오

```
1  #include <stdio.h>
2
3  int main() {
4      // stdout: 버퍼에 저장
5      printf("프로그램 시작...");
6      // stderr: 즉시 출력 ✅
7      fprintf(stderr, "위험한 작업 시작\n");
8      // 여기서 크래시 발생
9      int* p = NULL;
10     *p = 42; // ❌
11     // printf 내용은 버퍼에 남아 유실됨
12     // fprintf(stderr) 내용은 이미 출력됨
13 }
```

실행 결과:

위험한 작업 시작 ← stderr (이미 출력됨)
"프로그램 시작..."은 출력되지 않음

에러 출력 함수

```
1  // C
2  fprintf(stderr, "Error: %s\n", msg);
3
4  // perror: errno를 자동 해석
5  perror("fopen 실패");
6  // → "fopen 실패: No such file or directory"
```

C++ std::cerr VS std::clog

```
1  #include <iostream>
2
3  // cerr: 언버퍼드 — 즉시 출력 (에러용)
4  std::cerr << "치명적 오류: 메모리 부족\n";
5
6  // clog: 버퍼링 있음 — 성능 우선 (로그용)
7  std::clog << "[DEBUG] 함수 진입\n";
```

스트림

버퍼링

언제 사용

std::cout

있음

정상 출력

std::cerr

없음

에러, 경고

std::clog

있음

진단, 로그

```
# stdout과 stderr를 분리해서 처리
./prog > result.txt 2> error.txt
```

```
# stdout + stderr 같은 파일로
./prog > all.txt 2>&1
```

버퍼 플러시 제어

일반적으로 `"\n"` 을 사용하고, 즉시 출력이 반드시 필요한 경우에만 `std::endl` 또는 `fflush` 를 쓴다.

`fflush()` — 강제 플러시

```
1  #include <stdio.h>
2
3  int main() {
4      printf("진행 중...");
5      fflush(stdout);    // ← 즉시 화면에 출력
6      do_heavy_work();
7      printf(" 완료\n");
8  }
9
10 // fflush(NULL) — 모든 출력 스트림 플러시
11 fflush(NULL);
```

`\n` 활용 (라인 버퍼링 환경)

```
1  // 터미널에서는 \n이 즉시 플러시를 유발
2  printf("1단계 완료\n"); // ✅ 즉시 출력 (터미널)
3  printf("2단계 완료\n"); // ✅ 즉시 출력 (터미널)
4
5  // 파이프/파일에서는 \n도 즉시 플러시 안 함
6  // → setvbuf 또는 fflush 필요
```

C++ `std::endl` vs `"\n"`

```
1  #include <iostream>
2
3  // endl = '\n' 출력 + flush() 호출
4  std::cout << "결과: " << 42 << std::endl;
5  // ⚠️ flush가 포함 — 성능 저하
6
7  // "\n" = '\n' 출력만
8  std::cout << "결과: " << 42 << "\n";
9  // ✅ 빠름 — 버퍼링 그대로 유지
```

성능 비교

```
1  // ❌ 느림 — 매 줄마다 flush
2  for (int i = 0; i < 100000; i++) {
3      std::cout << i << std::endl;
4  }
5
6  // ✅ 빠름 — 버퍼링 활용
7  for (int i = 0; i < 100000; i++) {
8      std::cout << i << "\n";
9  }
10 // 마지막에 한 번 flush (또는 자동)
```

stdin 버퍼 잔류 문제

scanf 와 getchar / getline 을 혼용할 때 **버퍼에 남은 \n** 때문에 예상치 못한 동작이 발생한다.

문제 상황: C

```
1  #include <stdio.h>
2
3  int main() {
4      int age;
5      char name[32];
6
7      scanf("%d", &age);
8      // 입력: "25\n"
9      // scanf는 "25"만 읽고 '\n'은 버퍼에 남김
10
11     scanf("%c", &name[0]);
12     // '\n'을 읽어버림! 의도한 문자가 아님
13     printf("age=%d, name[0]=%d\n", age, name[0]);
14     // → age=25, name[0]=10 (10 = ASCII '\n')
15 }
```

문제 상황: C++

```
1  #include <iostream>
2  #include <string>
3
4  int main() {
5      int age;
6      std::string name;
7
8      // C++ 에서도 동일한 문제 발생
9      std::cin >> age;
10     // 입력: "25\n"
11     // age = 25, 버퍼에 '\n' 남음
12     std::getline(std::cin, name); // '\n'만 읽고 끝남
13     // name = "" ← 의도와 다름
14 }
```

stdin 잔류 문제 — 실전 권장 패턴

줄 전체를 먼저 읽고 파싱하면 잔류 자체가 발생하지 않는다.

C: `fgets` + `sscanf`

```
1  #include <stdio.h>
2
3  int main(void) {
4      char line[256];
5      int age;
6      char name[64];
7
8      // fgets: 줄 전체 소비 - '\n'이 버퍼에 남지 않음
9      if (fgets(line, sizeof(line), stdin))
10         sscanf(line, "%63s", name);
11
12     if (fgets(line, sizeof(line), stdin))
13         sscanf(line, "%d", &age);
14
15     printf("이름: %s, 나이: %d\n", name, age);
16     return 0;
17 }
```

`fgets` 는 `\n` 을 포함해 줄 끝까지 읽으므로
버퍼가 항상 깨끗하게 비워진다.

C++: `getline` + `istringstream`

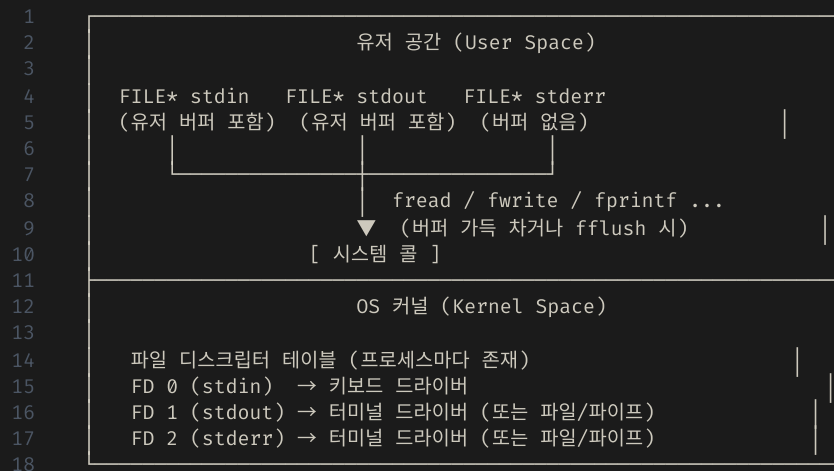
```
1  #include <iostream>
2  #include <sstream>
3  #include <string>
4
5  int main() {
6      std::string line;
7      std::string name;
8      int age;
9
10     // getline: 줄 전체 소비 - '\n'이 버퍼에 남지 않음
11     if (std::getline(std::cin, line))
12         std::istringstream(line) >> name;
13
14     if (std::getline(std::cin, line))
15         std::istringstream(line) >> age;
16
17     std::cout << "이름: " << name << ", 나이: " << age << "\n";
18 }
```

모든 입력을 `getline` 으로 통일하면

`ignore()` 나 `getchar()` 없이도 혼용 문제가 원천 차단된다.

파일 디스크립터 — 두 개의 레이어

C/C++ I/O는 **유저 공간 레이어**와 **OS 레이어** 두 겹으로 이루어진다.



`FILE*` 은 파일 디스크립터(정수)를 감싼(wrap) 구조체다.

유저 공간에서 버퍼를 관리하고, OS에 넘길 때만 시스템 콜을 호출한다.

FILE* 과 파일 디스크립터의 관계

FILE* 구조체 안에는 파일 디스크립터 번호가 포함되어 있다.

FILE* 내부 구조 (개념)

```
1 FILE* stdout 구조체
```

```
2  
3 fd = 1  
4 buf_start = 0x...  
5 buf_end = 0x...  
6 write_pos = 0x...  
7 ...  
8
```

← 파일 디스크립터 번호
← 유저 버퍼 시작
← 유저 버퍼 끝
← 현재 쓰기 위치

fileno() — FD 번호 확인

```
1 #include <stdio.h>  
2  
3 printf("%d\n", fileno(stdin)); // 0  
4 printf("%d\n", fileno(stdout)); // 1  
5 printf("%d\n", fileno(stderr)); // 2
```

fileno() 는 C 표준 함수로 모든 플랫폼에서 동작한다.
FD 번호 0 / 1 / 2 는 OS에 관계없이 동일하게 예약되어 있다.

리다이렉션과 파일 디스크립터

셸의 `>`, `<`, `|` 기호는 FD가 가리키는 대상을 바꿔 스트림 방향을 전환한다.

⚠ 주의: Windows 환경에서는 스트림 리다이렉션을 **커맨드 프롬프트(cmd)**에서 수행하세요. 커맨드 프롬프트에서는 POSIX 계열 운영체제와 유사하게 동작합니다. PowerShell에서는 리다이렉션 문법이 다르므로, 다음 예제가 그대로 동작하지 않을 수 있습니다.

셸 리다이렉션

```
# FD 1(stdout) → 파일
./prog > out.txt

# FD 0(stdin) ← 파일
./prog < input.txt

# FD 2(stderr) → 파일
./prog 2> error.txt

# FD 1과 FD 2 모두 같은 파일로
./prog > out.txt 2>&1

# FD 1 → 파이프 → 다음 프로그램의 FD 0
./prog | ./next
```

프로그램 코드는 변경 없이 `printf` / `scanf` 를 그대로 쓴다.

OS가 FD의 연결 대상만 바꾸면 데이터 흐름이 달라진다.

프로그램 내 스트림 분리 활용

```
1  #include <iostream>
2  #include <string>
3
4  int main() {
5      std::string name;
6      int score;
7
8      while (std::cin >> name >> score) {
9          if (score < 0 || score > 100) {
10             // 잘못된 데이터 → stderr
11             std::cerr << "[오류] " << name
12                 << ": 범위 초과 " << score << "\n";
13         } else {
14             // 정상 데이터 → stdout
15             std::cout << name << " "
16                 << (score >= 60 ? "합격" : "불합격") << "\n";
17         }
18     }
19 }
```

```
./grader < scores.txt          # 터미널에 모두 출력
./grader < scores.txt > result.txt # 정상 결과만 파일로
./grader < scores.txt 2> err.txt  # 에러만 파일로
```

요약

버퍼링

stdout (터미널) : 라인 버퍼링 → `\n` 또는 `fflush(stdout)` 로 플러시
stdout (파일/파이프): 풀 버퍼링 → 버퍼가 차거나 프로그램 종료 시 플러시
stderr : 언버퍼드 → 항상 즉시 출력 (크래시에도 안전)
stdin : 라인 버퍼링 → Enter 입력 시 프로그램에 전달

버퍼 제어

```
// C
fflush(stdout); // 즉시 플러시
setvbuf(stdout, NULL, _IONBF, 0); // 언버퍼드로 변경
```

```
// C++
std::cout << "... " << "\n"; // ✅ 빠름
std::cout << "... " << std::endl; // ⚠️ flush 포함 - 루프 안에서 주의
```

stdin 잔류 문제

```
// C
scanf("%d", &n); getchar(); // '\n' 제거 후 다음 입력
scanf(" %c", &c); // 포맷 앞 공백으로 해결
```

```
1 // C++
2 std::cin >> n; std::cin.ignore(); // '\n' 제거
3 std::getline(std::cin, s); // ✅ 한 줄 입력
```

파일 디스크립터와의 관계

```
1 FILE* (유저 공간, 버퍼링 있음)
2   ↳ fileno() / fdopen() / freopen() ← C 표준, 모든 플랫폼 동작
3 파일 디스크립터 0 / 1 / 2 (OS 커널, 버퍼링 없음)
4   ↳ 쉘 리다이렉션 (>, <, 2>, |)
5 실제 디바이스: 키보드 / 터미널 / 파일 / 파이프
```